

Trabalho Prático 2

Pareamento de Sequências de DNA

DCE797 – AEDS 3 | UNIFAL-MG

Isabela M. Andrade Isadora R. Grandaux Júlia A. da Rocha Otávio de Oliveira
Pedro Brassi Luccas

Bacharelado em Ciência da Computação – UNIFAL

Alfenas, Abril de 2026

- 1 Introdução
- 2 Metodologia
- 3 Resultados
- 4 Análise Crítica

O Problema de Alinhamento de DNA

Definição

Dadas duas sequências de bases **A, T, C, G**, alinhar inserindo **lacunas (-)** para **maximizar a pontuação** total.

Regras de pontuação:

- **Par complementar** (A-T, C-G): **+2 pontos**
- **Par não complementar**: **-1 ponto**
- **Base com lacuna**: **-2 pontos**

Problema central da **Bioinformática**:
comparação genética e análise evolutiva.

Exemplo de alinhamento

Antes (score: -2)

ACACACTA
TCGTGTGT

Depois (score: +10)

A-CACACTA
TCGTGTG-T

Dois Paradigmas Implementados

Algoritmo Guloso

- Percorre as sequências **uma única vez**
- Escolhe a **melhor opção local** a cada passo
- Complexidade: $O(n + m)$ tempo, $O(1)$ espaço
- Rápido, mas **não garante ótimo global**

Programação Dinâmica

- Matriz $dp[i][j]$: melhor pontuação para prefixos $s_1[0..i]$ e $s_2[0..j]$
- Considera **todas as combinações** possíveis
- Complexidade: $O(nm)$ tempo e espaço
- **Garante solução ótima**

Recorrência da PD:

$$dp[i][j] = \max \begin{cases} dp[i-1][j-1] + \text{score}(s_1[i], s_2[j]) & \text{alinhar} \\ dp[i-1][j] - 2 & \text{lacuna em } s_2 \\ dp[i][j-1] - 2 & \text{lacuna em } s_1 \end{cases}$$

Comparação: preenchimento bottom-up x traceback

Comparação: preenchimento bottom-up x traceback | score = 10

(A) Bottom-up (ordem de preenchimento)

	-	T	C	G	T	G	T	G	T
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2 ₁	0 ₂	-2 ₃	-4 ₄	-6 ₅	-8 ₆	-10 ₇	-12 ₈
C	-4	0 ₉	1 ₁₀	2 ₁₁	0 ₁₂	-2 ₁₃	-4 ₁₄	-6 ₁₅	-8 ₁₆
A	-6	-2 ₁₇	-1 ₁₈	0 ₁₉	4 ₂₀	2 ₂₁	0 ₂₂	-2 ₂₃	-4 ₂₄
C	-8	-4 ₂₅	-3 ₂₆	1 ₂₇	2 ₂₈	6 ₂₉	4 ₃₀	2 ₃₁	0 ₃₂
A	-10	-6 ₃₃	-5 ₃₄	-1 ₃₅	3 ₃₆	4 ₃₇	8 ₃₈	6 ₃₉	4 ₄₀
C	-12	-8 ₄₁	-7 ₄₂	-3 ₄₃	1 ₄₄	5 ₄₅	6 ₄₆	10 ₄₇	8 ₄₈
T	-14	-10 ₄₉	-9 ₅₀	-5 ₅₁	-1 ₅₂	3 ₅₃	4 ₅₄	8 ₅₅	9 ₅₆
A	-16	-12 ₅₇	-11 ₅₈	-7 ₅₉	-3 ₆₀	1 ₆₁	5 ₆₂	6 ₆₃	10 ₆₄

Número pequeno = ordem bottom-up
(1 → 64) | valores = dp[i][j]

(B) Traceback (caminho ótimo)

	-	T	C	G	T	G	T	G	T
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4	0	1	0	-2	-4	-6	-8	-10
A	-6	-2	-1	0	2	0	-2	-4	-6
C	-8	-4	-3	1	2	4	2	0	-2
A	-10	-6	-5	-1	3	4	6	4	2
C	-12	-8	-7	-3	1	5	6	8	6
T	-14	-10	-9	-5	-1	3	4	9	10
A	-16	-12	-11	-7	-3	1	5	6	10

Setas:

■ MATCH (A-T, C-G)

■ MISMATCH

■ Preenchimento (um ótimo):

A-C-A-C-A-T-T-A
T-C-G-T-G-T-G

Código – score(a, b)

```
if ((a == 'A' && b == 'T') ||  
    (a == 'T' && b == 'A') ||  
    (a == 'C' && b == 'G') ||  
    (a == 'G' && b == 'C'))  
    return 2;  
if (a == '-' || b == '-')  
    return -2;  
return -1;
```

Utilizada por **ambos** os algoritmos, garantindo consistência na avaliação das soluções.

Regras implementadas

- **A-T, T-A, C-G, G-C** → +2
- **Lacuna ('-')** em qualquer posição → -2
- **Demais pares** → -1

Ponto importante

A verificação da lacuna é feita **antes** do retorno padrão -1, evitando classificar lacuna como mismatch.

Inicialização e laço principal

```
int i = 0, j = 0;
long int pontuacao = 0;
int n = strlen(s1);
int m = strlen(s2);

while (i < n || j < m) {
    int match = -9999;
    int gap_s1 = -9999;
    int gap_s2 = -9999;

    if (i < n && j < m)
        match = score(s1[i], s2[j]);
    if (i < n) gap_s1 = GAP;
    if (j < m) gap_s2 = GAP;
```

Escolha gulosa

```
    if (match >= gap_s1 &&
        match >= gap_s2) {
        pontuacao += match;
        i++; j++;
    }
    else if (gap_s1 >= gap_s2) {
        pontuacao += gap_s1;
        i++;
    }
    else {
        pontuacao += gap_s2;
        j++;
    }
}
```

Inicializar com -9999 evita escolhas inválidas quando um índice chegou ao fim. Mismatch (-1) é preferido

Alocação e casos base

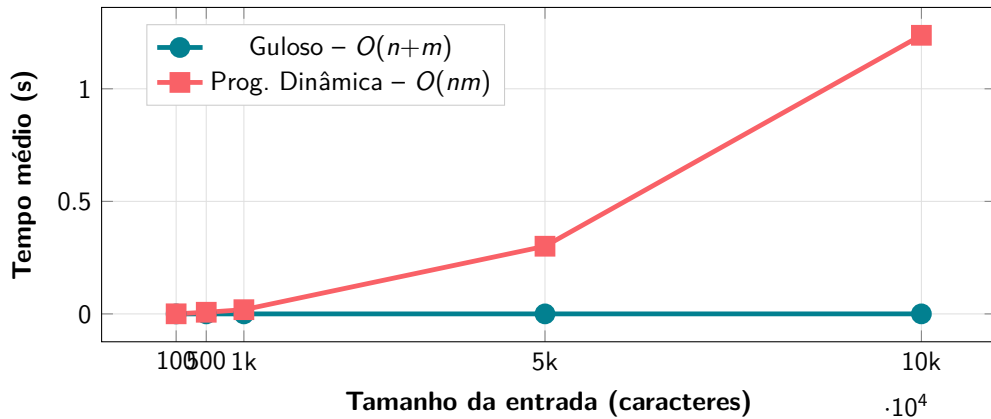
```
long int **dp =  
    malloc((n+1)*sizeof(long int*));  
for (int i = 0; i <= n; i++)  
    dp[i] = malloc((m+1)*  
        sizeof(long int));  
  
dp[0][0] = 0;  
for (int i = 1; i <= n; i++)  
    dp[i][0] = dp[i-1][0] + GAP;  
for (int j = 1; j <= m; j++)  
    dp[0][j] = dp[0][j-1] + GAP;
```

Recorrência e resultado

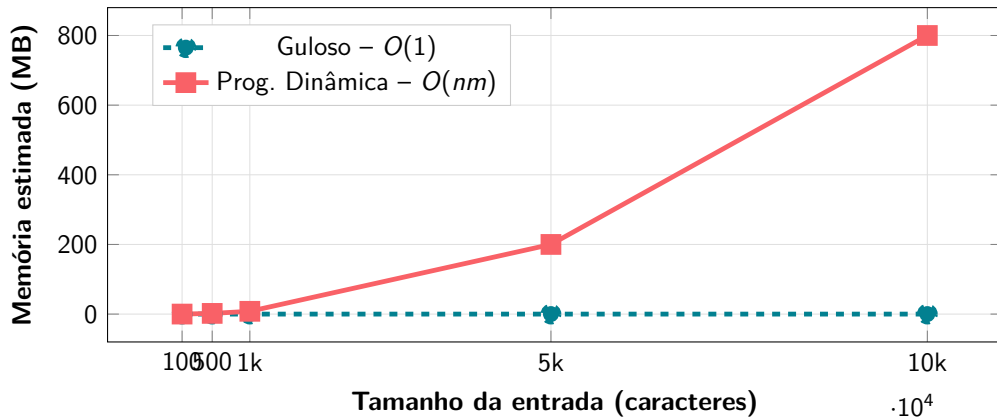
```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= m; j++) {  
        long int diag =  
            dp[i-1][j-1] +  
            score(s1[i-1], s2[j-1]);  
        long int cima =  
            dp[i-1][j] + GAP;  
        long int esq =  
            dp[i][j-1] + GAP;  
        dp[i][j] =  
            maiorTres(diag, cima, esq);  
    }  
}  
return dp[n][m];
```

Liberação: `free(dp[i])` em loop + `free(dp)`. A resposta ótima está em `dp[n][m]`.

Tempo de Execução vs. Tamanho da Entrada



Consumo de Memória vs. Tamanho da Entrada



Para $n = 10.000$: $8 \times (10001)^2 \approx 800$ MB só na matriz dp .

Tabela de Pontuações – Médias por Tamanho

Tamanho	Guloso (pont.)	Guloso (tempo s)	PD (pont.)	PD (tempo s)
100	-24,4	0,000004	35,6	0,000403
500	-129,2	0,000009	196,6	0,007334
1000	-281,8	0,000016	408,2	0,018722
5000	-1253,0	0,000083	2174,2	0,300653
10000	-2525,8	0,000142	4443,2	1,237715

- PD obteve pontuação **superior em 100% das instâncias**
- Guloso produziu pontuações **negativas** em todos os tamanhos

Guloso – Vantagens

- Execução **muito rápida** mesmo para 10.000 bases
- Memória **constante** $O(1)$
- Ideal quando velocidade é prioritária

Guloso – Limitações

- Pontuações **negativas** em todas as instâncias
- Decisões locais ignoram ganhos futuros

Prog. Dinâmica – Vantagens

- **Garante o ótimo global** em 100% dos casos
- Pontuações sempre positivas e maiores

Prog. Dinâmica – Limitações

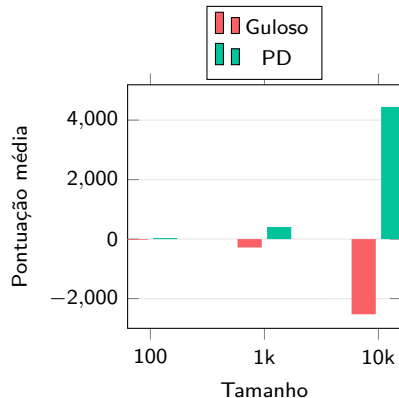
- Tempo **quadrático**: lento para entradas muito grandes
- Memória **quadrática**: ≈ 800 MB para $n = 10.000$

Principais Resultados

- **PD** garante alinhamento ótimo em $O(nm)$, com pontuações sempre positivas
- **Guloso** oferece velocidade $O(n+m)$ com perda de qualidade
- 25 instâncias confirmaram o comportamento teórico

Quando usar cada um?

- **Guloso**: resposta rápida, baixo custo computacional
- **PD**: otimalidade essencial, recursos disponíveis



Muito Obrigado!

Dúvidas e perguntas

Disciplina: DCE797 – AEDS 3

Professor: Iago Augusto de Carvalho

Grupo: Isabela, Isadora, Júlia, Otávio, Pedro

UNIFAL-MG – Alfenas, Abril de 2026